Merlette — Spécifications techniques par user story

Une section par story de `01-user-stories.md`, même numéro.

Saisie dans Taiga

1. Ouvrir la user story (ou la créer avec le titre indiqué).
2. Coller le bloc **Spec** dans la description.
3. *Tâches* → *Ajout en masse* : coller le bloc **Tâches** (une tâche par ligne).

Préfixes : `[BACK]` API · `[FRONT]` interface · `[BD]` base de données · `[TEST]` Postman. Stack : React + Vite · Node.js + Express · PostgreSQL · JWT + bcrypt · Multer.

Socle technique (story technique, Sprint 1)

Spec

- Monorepo `client/` (React) + `server/` (Express).
- `.env` : `PORT`, `DATABASE_URL`, `JWT_SECRET`.
- Erreurs JSON : `{ error: { code, message, details } }`.
- `GET /api/health` → 200.

Tâches

```
[BD] Créer la base et exécuter schema.sql (tables + seed)
[BACK] Initialiser Express (CORS, JSON, dossier /uploads statique)
[BACK] Connexion PostgreSQL et lecture du .env
[BACK] Middleware de gestion d'erreurs au format uniforme
[BACK] Route GET /api/health
[FRONT] Initialiser React + Vite, routeur et client HTTP avec token
[FRONT] Layout : barre de navigation, cloche, bouton + Observation
```

#11 — Créer un compte · Must · 2 pts

Spec

- API : `POST /api/auth/register` → 201 `{token, user}`
- BD : `users`
- Règles : username 3–30 `[a-zA-Z0-9_]`, email valide, mot de passe ≥ 8 ; bcrypt coût 10 ; 409 si email/pseudo pris.
- Tests : T01–T07

Tâches

```
[BACK] Route POST /auth/register avec validation
[BACK] Hachage bcrypt et génération du JWT (24 h)
[FRONT] Page /register avec affichage des erreurs
[FRONT] Connexion automatique après inscription
[TEST] Scénarios Postman T01–T07
```

#10 — Se connecter et retrouver mes conversations · Must · 3 pts

Spec

- API : `POST /api/auth/login`, `GET /api/auth/me`, `GET /api/users/me/conversations`
- BD : `users`, `comments`, `proposals`, `observations`
- Règles : 401 `INVALID_CREDENTIALS` (même message email/mot de passe) ; JWT dans `Authorization: Bearer` ; discussions = observations publiées, commentées ou avec proposition, triées par dernière activité.
- Tests : T08–T14, T73–T74

Tâches

```
[BACK] Route POST /auth/login (bcrypt.compare + JWT)
[BACK] Middlewares requireAuth / optionalAuth / requireAdmin
[BACK] Route GET /auth/me
[BACK] Route GET /users/me/conversations (requête UNION)
[FRONT] Page /login + stockage du token + déconnexion
[FRONT] Page /me/conversations
[TEST] Scénarios Postman T08–T14, T73–T74
```

#12 — Consulter et modifier mon profil · Should · 2 pts

Spec

- API : `GET /api/users/:id`, `PATCH /api/users/me`
- BD : `users`
- Règles : profil public sans email ; stats = nb observations, nb propositions ; seuls `bio` et `avatarUrl` modifiables.
- Tests : T76–T77

Tâches

```
[BACK] Route GET /users/:id avec statistiques
[BACK] Route PATCH /users/me
[FRONT] Page /users/:id + formulaire d'édition si c'est moi
[TEST] Scénarios Postman T76–T77
```

#1 — Poster des images · Must · 5 pts

Spec

- API : `POST /api/observations` (multipart, champ `photos`), `POST /api/observations/:id/photos`
- BD : `observations`, `photos`
- Règles : JWT requis ; 1–5 images JPEG/PNG/WEBP ≤ 5 Mo ; insertion en transaction ; fichiers supprimés si échec ; statut initial `NEEDS_ID`.
- Tests : T21–T28, T36

Tâches

```
[BACK] Configurer Multer (types, 5 Mo, 5 fichiers max)
[BACK] Route POST /observations (transaction observation + photos)
[BACK] Suppression des fichiers uploadés en cas d'erreur
[BACK] Route POST /observations/:id/photos (auteur uniquement, max 5 au total)
[FRONT] Page /observations/new : sélection et aperçu des photos
[TEST] Scénarios Postman T21–T28, T36
```

#13 — Décrire mon observation · Must · 2 pts

Spec

- API : champs de `POST /api/observations`
- BD : `observations` (`category_id`, `observed_at`, `title`, `description`, `location_name`, `latitude`, `longitude`)
- Règles : catégorie et date obligatoires ; date ≤ aujourd'hui (aussi en `CHECK` SQL).
- Tests : T24–T26

Tâches

```
[BACK] Validation des champs (catégorie existante, date non future, lat/long)
[FRONT] Champs du formulaire : catégorie, date, lieu, description, titre
[TEST] Scénarios Postman T24–T26
```

#14 — Gérer mes observations · Should · 3 pts

Spec

- API : `GET /api/users/me/observations`, `PATCH /api/observations/:id`, `DELETE /api/observations/:id`, `DELETE /api/observations/:id/photos/:photoId`
- BD : `observations`, `photos` (suppression en cascade)
- Règles : auteur uniquement (sinon 403) ; suppression = cascade BD + fichiers ; au moins 1 photo restante.
- Tests : T34–T35, T75, T80, T82

Tâches

```
[BACK] Route GET /users/me/observations
[BACK] Route PATCH /observations/:id (contrôle auteur)
[BACK] Route DELETE /observations/:id (cascade + fichiers)
[BACK] Route DELETE /observations/:id/photos/:photoId
[FRONT] Page /me/observations avec statut
[FRONT] Modifier / supprimer avec confirmation
[TEST] Scénarios Postman T34–T35, T75, T80, T82
```

#15 — Voir le détail d'une observation · Must · 3 pts

Spec

- API : `GET /api/observations/:id`
- BD : `observations`, `photos`, `proposals`, `votes`, `species`, `categories`, `users`
- Règles : public ; `myVoteProposalId` si connecté ; 404 si inexistante.
- Tests : T32–T33

Tâches

```
[BACK] Route GET /observations/:id (photos, propositions, votes, auteur)
[FRONT] Page /observations/:id : galerie, infos, statut, espèce retenue
[TEST] Scénarios Postman T32–T33
```

#8 — Voir les demandes d'identification récentes · Must · 2 pts

Spec

- API : `GET /api/observations?status=&sort=recent&page=&limit=`
- BD : `observations` (index `created_at DESC`)
- Règles : tri date de publication décroissante ; 20 par page (max 50) ; réponse `{data, pagination}`.
- Tests : T29

Tâches

```
[BACK] Route GET /observations avec pagination et filtre status
[FRONT] Page d'accueil : grille de cartes (photo, catégorie, statut, auteur, compteurs)
[FRONT] Pagination et filtre « À identifier »
[TEST] Scénario Postman T29
```

#9 — Filtrer par catégorie · Must · 2 pts

Spec

- API : `GET /api/categories`, `GET /api/observations?category=<slug>`
- BD : `categories` (8 catégories en seed)
- Règles : filtre combinable avec statut ; filtre visible dans l'URL du front.
- Tests : T15, T30–T31, T55

Tâches

```
[BACK] Route GET /categories
[BACK] Filtre category (slug) sur GET /observations
[FRONT] Sélecteur de catégorie synchronisé avec l'URL
[TEST] Scénarios Postman T15, T30–T31, T55
```

#16 — Rechercher une espèce · Should · 2 pts

Spec

- API : `GET /api/species?q=&category=&limit=`, `GET /api/species/:id`
- BD : `species` (30 espèces en seed, extension `unaccent`)
- Règles : ≥ 2 caractères (sinon 400) ; insensible à la casse et aux accents ; nom commun ou scientifique.
- Tests : T16–T18

Tâches

```
[BACK] Route GET /species avec recherche ILIKE + unaccent
[FRONT] Champ d'autocomplétion (déclenché à 2 caractères)
[TEST] Scénarios Postman T16–T18
```

#17 — Proposer une espèce · Must · 3 pts

Spec

- API : `POST /api/observations/:id/proposals {speciesId}`, `DELETE /api/proposals/:id`
- BD : `proposals` (`UNIQUE (observation_id, species_id)`), `votes`
- Règles : proposition = vote du proposeur (upsert) ; 409 si espèce déjà proposée ; notification `NEW_PROPOSAL` à l'auteur ; recalcul du consensus ; suppression seulement si aucun autre vote.
- Tests : T37–T41, T46, T56

Tâches

```
[BACK] Route POST /observations/:id/proposals (proposition + vote en transaction)
[BACK] Route DELETE /proposals/:id
[FRONT] Bouton « Proposer » + autocomplétion espèce
[TEST] Scénarios Postman T37–T41, T46, T56
```

#4 — Voter sur l'espèce · Must · 3 pts

Spec

- API : `PUT /api/observations/:id/vote {proposalId}`, `DELETE /api/observations/:id/vote`
- BD : `votes` (`UNIQUE (observation_id, user_id)`, FK composite vers `proposals`)
- Règles : 1 vote par utilisateur et par observation (upsert) ; consensus = meilleure proposition ≥ 3 votes **et** ≥ 2/3 → `IDENTIFIED`, sinon `NEEDS_ID` ; notification `OBSERVATION_IDENTIFIED` au passage à `IDENTIFIED`.
- Tests : T42–T44, T48–T54

Tâches

```
[BACK] Service ConsensusService.recompute(observationId)
[BACK] Route PUT /observations/:id/vote (upsert + recalcul)
[BACK] Route DELETE /observations/:id/vote (+ recalcul)
[FRONT] Boutons Voter / Retirer mon vote, mise à jour du statut
[TEST] Scénarios Postman T42–T44, T48–T54
```

#5 — Consulter les votes · Must · 2 pts

Spec

- API : `GET /api/observations/:id/proposals`
- BD : `proposals`, `votes`, `species`, `users`
- Règles : public ; par proposition : votes, %, votants ; tri votes décroissant puis ancienneté ; objet `consensus`.
- Tests : T45, T47, T49

Tâches

```
[BACK] Route GET /observations/:id/proposals (agrégation des votes)
[FRONT] Liste des propositions avec barre de %, votants, mon vote surligné
[TEST] Scénarios Postman T45, T47, T49
```

#3 — Commenter les posts · Must · 2 pts

Spec

- API : `GET /api/observations/:id/comments`, `POST /api/observations/:id/comments {content}`

- BD : `comments`
- Règles : JWT requis pour écrire ; 1–1000 caractères, non vide ; ordre chronologique ; notification `NEW_COMMENT` à l'auteur + anciens commentateurs (sauf soi).
- Tests : T57–T62

Tâches

```
[BACK] Route GET /observations/:id/comments
[BACK] Route POST /observations/:id/comments (+ notifications)
[FRONT] Fil de commentaires + champ de saisie
[TEST] Scénarios Postman T57–T62
```

#18 — Modifier / supprimer mes commentaires · Could · 1 pt

Spec

- API : `PATCH /api/comments/:id`, `DELETE /api/comments/:id`
- BD : `comments` (`updated_at`)
- Règles : auteur (ou admin pour suppression) sinon 403 ; `edited = updated_at > created_at`.
- Tests : T63–T64, T78–T79

Tâches

```
[BACK] Routes PATCH et DELETE /comments/:id (contrôle auteur/admin)
[FRONT] Actions Modifier / Supprimer + mention « modifié »
[TEST] Scénarios Postman T63–T64, T78–T79
```

#6 — Être notifié du déroulement de l'identification · Must · 3 pts

Spec

- API : `GET /api/notifications?unread=&page=`, `GET /api/notifications/unread-count`, `PATCH /api/notifications/:id/read`, `PATCH /api/notifications/read-all`
- BD : `notifications` (index `user_id, is_read, created_at`)
- Règles : types `NEW_PROPOSAL`, `NEW_COMMENT`, `OBSERVATION_IDENTIFIED` ; jamais pour ses propres actions ; polling front 30 s ; 404 si la notification n'est pas à moi.
- Tests : T65–T72

Tâches

```
[BACK] Service NotificationService.notify(destinataires, type, observation, acteur)
[BACK] Routes GET /notifications et /notifications/unread-count
[BACK] Routes PATCH /notifications/:id/read et /notifications/read-all
[FRONT] Badge cloche avec polling 30 s
[FRONT] Liste des notifications (clic = lue + ouvre l'observation)
[TEST] Scénarios Postman T65–T72
```

#19 — Modérer le contenu · Could · 2 pts

Spec

- API : `DELETE /api/observations/:id`, `DELETE /api/comments/:id` (rôle `ADMIN` autorisé)
- BD : `users.role`
- Règles : admin peut supprimer tout contenu ; compte admin en seed.
- Tests : T81

Tâches

```
[BACK] Autoriser le rôle ADMIN dans les contrôles de suppression
[FRONT] Boutons de suppression visibles pour l'admin
[TEST] Scénario Postman T81
```

#20 — Gérer le référentiel · Could · 3 pts

Spec

- API : `POST /api/categories`, `POST /api/species`
- BD : `categories`, `species`
- Règles : admin uniquement (sinon 403) ; 409 si nom/slug/nom scientifique déjà pris.
- Tests : T19–T20

Tâches

```
[BACK] Routes POST /categories et POST /species (requireAdmin)
[FRONT] Page /admin : formulaires catégorie et espèce
[TEST] Scénarios Postman T19–T20
```